

JWT (JSON Web Token) - Practical Implementation Guide

What is JWT?

JWT is a compact, URL-safe token format used for securely transmitting information between parties as a JSON object. It's commonly used for authentication and authorization in modern web applications.

JWT Structure

A JWT consists of three parts separated by dots (`.`):

```
header.payload.signature
```

1. Header

Contains the token type and hashing algorithm:

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

2. Payload

Contains claims (user data and metadata):

```
{
  "sub": "1234567890",
  "name": "John Doe",
  "iat": 1516239022,
  "exp": 1516242622
}
```

3. Signature

Created by encoding header + payload with a secret key:

```
HMACSHA256(
  base64UrlEncode(header) + "." + base64UrlEncode(payload),
  secret
)
```

Standard Claims

- **iss** (issuer): Who issued the token
- **sub** (subject): User identifier
- **aud** (audience): Intended recipient
- **exp** (expiration): Token expiration timestamp
- **iat** (issued at): Token creation timestamp
- **nbf** (not before): Token not valid before this time
- **jti** (JWT ID): Unique token identifier

Implementation Examples

Node.js with Express

Installation

```
npm install jsonwebtoken bcrypt express dotenv
```

1. Generate JWT (Login)

```
const jwt = require("jsonwebtoken");
const bcrypt = require("bcrypt");

// Login endpoint
app.post("/login", async (req, res) => {
  const { email, password } = req.body;

  // Find user in database
  const user = await User.findOne({ email });

  if (!user) {
    return res.status(401).json({ error: "Invalid credentials" });
  }

  // Verify password
  const validPassword = await bcrypt.compare(password, user.password);

  if (!validPassword) {
    return res.status(401).json({ error: "Invalid credentials" });
  }

  // Create JWT payload
  const payload = {
    id: user._id,
    email: user.email,
    role: user.role,
  };

  // Generate token
  const token = jwt.sign(payload, process.env.JWT_SECRET, { expiresIn: "24h" });

  res.json({
    token,
    user: { id: user._id, email: user.email },
  });
});
```

2. Verify JWT (Middleware)

```
const jwt = require("jsonwebtoken");

// Authentication middleware
const authenticateToken = (req, res, next) => {
  // Get token from header
  const authHeader = req.headers["authorization"];
  const token = authHeader && authHeader.split(" ")[1]; // Bearer TOKEN

  if (!token) {
    return res.status(401).json({ error: "Access token required" });
  }

  // Verify token
  jwt.verify(token, process.env.JWT_SECRET, (err, user) => {
    if (err) {
      return res.status(403).json({ error: "Invalid or expired token" });
    }

    // Attach user data to request
    req.user = user;
    next();
  });
};

// Protected route
app.get("/profile", authenticateToken, (req, res) => {
  res.json({ user: req.user });
});
```

3. Refresh Token Implementation

```

const jwt = require("jsonwebtoken");

// Store refresh tokens (use Redis in production)
let refreshTokens = [];

// Generate both tokens
app.post("/login", async (req, res) => {
  // ... authentication logic ...

  const accessToken = jwt.sign(payload, process.env.JWT_SECRET, {
    expiresIn: "15m",
  });

  const refreshToken = jwt.sign(payload, process.env.REFRESH_TOKEN_SECRET, {
    expiresIn: "7d",
  });

  refreshTokens.push(refreshToken);

  res.json({ accessToken, refreshToken });
});

// Refresh endpoint
app.post("/refresh", (req, res) => {
  const { refreshToken } = req.body;

  if (!refreshToken) {
    return res.status(401).json({ error: "Refresh token required" });
  }

  if (!refreshTokens.includes(refreshToken)) {
    return res.status(403).json({ error: "Invalid refresh token" });
  }

  jwt.verify(refreshToken, process.env.REFRESH_TOKEN_SECRET, (err, user) => {
    if (err) {
      return res.status(403).json({ error: "Invalid refresh token" });
    }

    const accessToken = jwt.sign(
      { id: user.id, email: user.email },
      process.env.JWT_SECRET,
      { expiresIn: "15m" }
    );

    res.json({ accessToken });
  });
});

// Logout
app.post("/logout", (req, res) => {
  const { refreshToken } = req.body;
  refreshTokens = refreshTokens.filter((token) => token !== refreshToken);
  res.json({ message: "Logged out successfully" });
});

```

Frontend Integration (JavaScript/React)

1. Login and Store Token

```
// Login function
async function login(email, password) {
  const response = await fetch("/api/login", {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
    },
    body: JSON.stringify({ email, password }),
  });

  const data = await response.json();

  if (response.ok) {
    // Store token in localStorage
    localStorage.setItem("token", data.token);
    return data;
  } else {
    throw new Error(data.error);
  }
}
```

2. Send Token with Requests

```
// Fetch with token
async function fetchProtectedData() {
  const token = localStorage.getItem("token");

  const response = await fetch("/api/profile", {
    method: "GET",
    headers: {
      Authorization: `Bearer ${token}`,
      "Content-Type": "application/json",
    },
  });

  return await response.json();
}
```

3. Axios Interceptor

```
import axios from "axios";

// Add token to all requests
axios.interceptors.request.use(
  (config) => {
    const token = localStorage.getItem("token");
    if (token) {
      config.headers.Authorization = `Bearer ${token}`;
    }
    return config;
  },
  (error) => Promise.reject(error)
);

// Handle token expiration
axios.interceptors.response.use(
  (response) => response,
  async (error) => {
    if (error.response.status === 401) {
      // Token expired - try to refresh
      const refreshToken = localStorage.getItem("refreshToken");

      try {
        const response = await axios.post("/api/refresh", { refreshToken });
        localStorage.setItem("token", response.data.accessToken);

        // Retry original request
        error.config.headers.Authorization = `Bearer ${response.data.accessToken}`;
        return axios.request(error.config);
      } catch (refreshError) {
        // Refresh failed - redirect to login
        localStorage.removeItem("token");
        localStorage.removeItem("refreshToken");
        window.location.href = "/login";
      }
    }
    return Promise.reject(error);
  }
);
```

Security Best Practices

1. Token Storage

- **localStorage:** Easy but vulnerable to XSS attacks
- **sessionStorage:** Cleared when tab closes
- **httpOnly Cookie:** Most secure, not accessible via JavaScript
- **Memory:** Most secure but lost on refresh

2. Token Configuration

```
// Use strong secrets
const token = jwt.sign(payload, process.env.JWT_SECRET, {
  algorithm: "HS256", // Use HS256 or RS256
  expiresIn: "15m", // Short expiration for access tokens
  issuer: "your-app-name",
  audience: "your-app-users",
});
```

3. Best Practices Checklist

- ☒ Use HTTPS only
- ☒ Set short expiration times (15-30 minutes)
- ☒ Implement refresh token mechanism
- ☒ Store secrets in environment variables
- ☒ Validate all claims (exp, iss, aud)
- ☒ Use strong, random secret keys
- ☒ Implement token blacklist for logout
- ☒ Never store sensitive data in payload
- ☒ Use httpOnly cookies when possible
- ☒ Implement rate limiting on auth endpoints

4. Environment Variables

```
# .env file
JWT_SECRET=your-super-secret-key-here-make-it-long-and-random
REFRESH_TOKEN_SECRET=another-different-secret-key
TOKEN_EXPIRY=15m
REFRESH_TOKEN_EXPIRY=7d
```

Common Pitfalls to Avoid

1. **Storing sensitive data in payload** - JWT is encoded, not encrypted
2. **Using weak secrets** - Use cryptographically secure random strings
3. **No token expiration** - Always set expiration time
4. **Not validating tokens properly** - Always verify signature and claims
5. **Storing tokens in localStorage without considering XSS** - Consider httpOnly cookies
6. **Not implementing refresh tokens** - Users shouldn't re-login frequently
7. **Not handling token expiration on frontend** - Implement automatic refresh

Testing JWT

Manual Testing with curl

```
# Login
curl -X POST http://localhost:3000/login \
  -H "Content-Type: application/json" \
  -d '{"email":"user@example.com","password":"password123"}'

# Access protected route
curl -X GET http://localhost:3000/profile \
  -H "Authorization: Bearer YOUR_TOKEN_HERE"
```

Decode JWT (jwt.io)

Visit <https://jwt.io> to manually decode and verify tokens during development.

Resources

- [JWT.io](https://jwt.io) - Decode and verify tokens
 - [RFC 7519](https://tools.ietf.org/html/rfc7519) - JWT specification
 - [OWASP JWT Cheat Sheet](#)
-